CADT
IDT វិទ្យាស្ថានបច្ចេកវិទ្យាឌីជីថល
Institute of Digital Technology

Cambodia Academy of Digital Technology
Institute of Digital Technology

Department: Computer Science
Specialization: Software Engineering
Course: Database Analysis and Design
Teach by: Thear Sophal & Yann Kimhuoy

ASSET MANAGEMENT DATABASE SYSTEM

FINAL PROJECT REPORT

Team Member:

- Khy Chhoungech
- Seng Namkea
- Ly Sophearak
- Thai Chansothymon
- Poch Seavmeng
- Phal Sambathoudom

Asset Management Database System is a system that is designed to make sure that every asset in a university is manageable, and it also increases security to keep our asset safe and maintainable

TABLE OF CONTENTS

I. INTRODUCTION

is designed to track, manage, and maintain physical resources within a school environment. These resources include buildings, classrooms, equipment, storage items, and records of usage, maintenance, and procurement.

1.1 Project Description

is designed to track, manage, and maintain physical resources within a school environment. These resources include buildings, classrooms, equipment, storage items, and records of usage, maintenance, and procurement.

1.2 Objective

- Centralized Data Management: Store all asset-related information in a single database system.
- Asset Tracking and Allocation: Track the location and status of each asset
- Maintenance Monitoring: Record and monitor repair history for assets.
- Procurement Management: Maintain detailed purchase records for financial tracking and auditing.
- User Accountability: Track employee interactions with assets via requisition and audit logs

II. DATABASE ANALYSIS

2.1 User Requirements

1. Asset Management:

- Add, update, delete, and view assets
- Assign assets to classrooms or storage rooms
- Categorize assets by the Type_Of_Asset.

2. Building & Classroom Management:

- Manage building records
- Associate classrooms with buildings
- Track which assets are located in each classroom.

3. Storage Management:

- Record storage rooms and their locations
- Track assets stored in each storage room.

4. Employee Management:

- Store employee details
- Maintain employee addresses
- Link employees to actions (repairs, audits, requisitions).

5. Purchase Record:

- Record asset purchases
- Store supplier details, cost, and purchase date.

6. Repair Record:

- Log repair activities for assets
- Track repair dates, costs, and responsible employees.

7. Borrow_Logging

- Allow employees to borrow assets from storage
- Record borrow and return dates
- Track asset availability

8. Audit Logging

- Record all critical operations (insert, update, delete) on each table
- Maintain timestamps for accountability

2.2 Project Requirements

1. Data Definition:

- Handle increasing number of assets
- Establish relationships
- Maintain normalization.

2. Data Manipulation:

- Insert new assets, employees, records...
- Update asset status, employee information, classroom details...
- Delete obsolete or incorrect records.

3. Data Query:

- Retrieve assets data
- Enable filtering, sorting, and grouping.

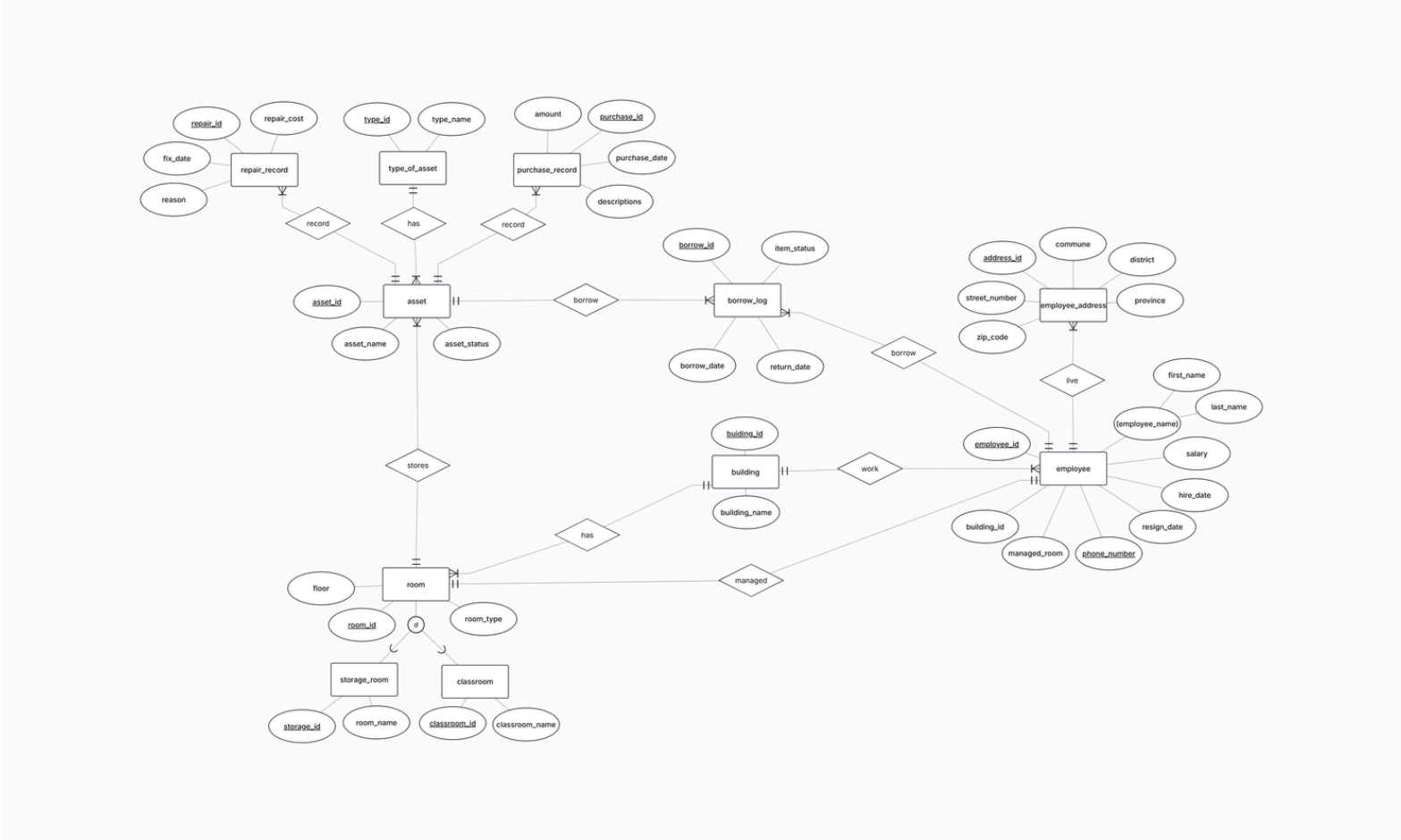
4. Data Integrity and Consistency:

- Enforce constraints
- Ensure minimal data loss with backup and recovery mechanisms
- Efficient query execution for large datasets.

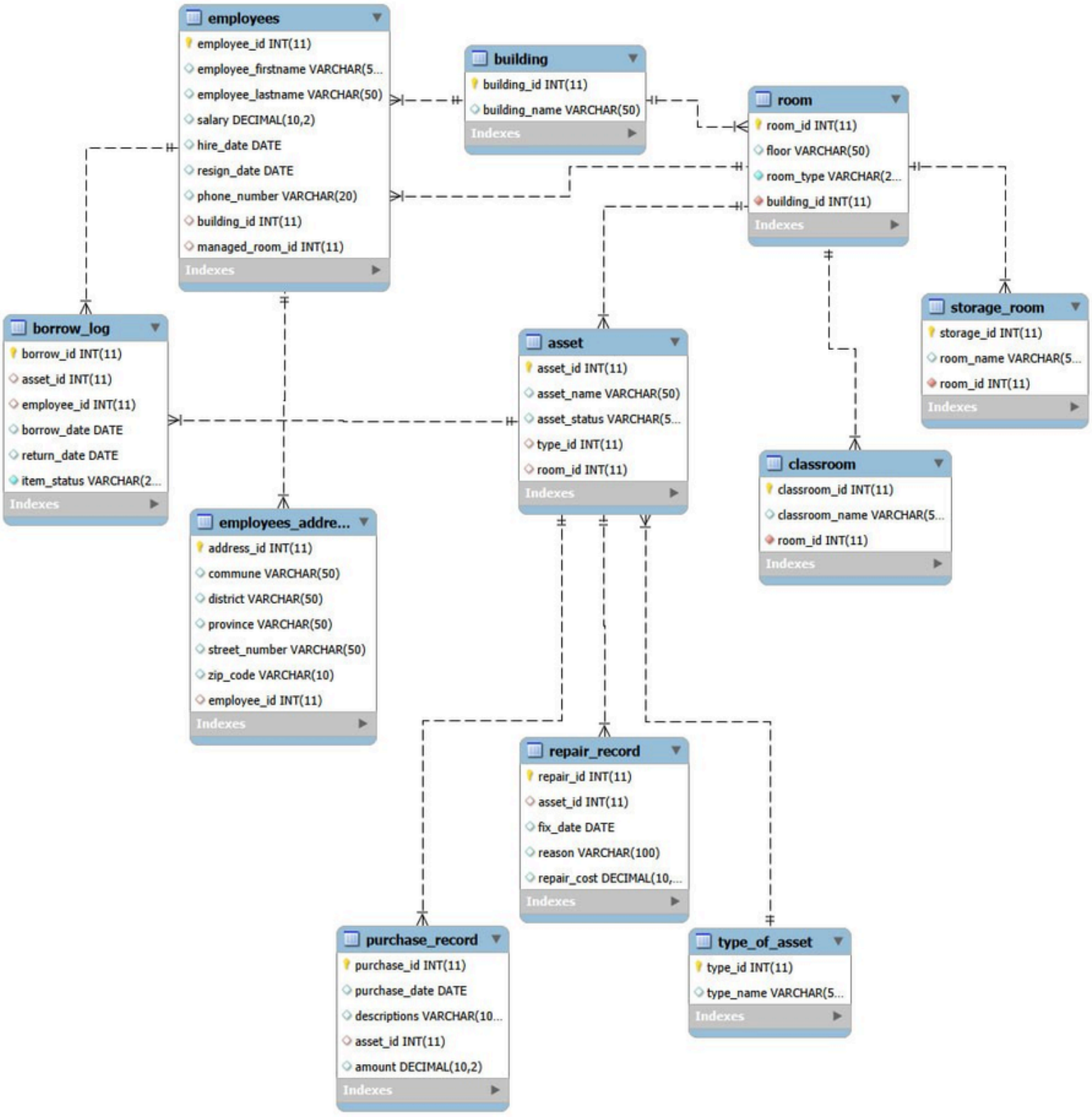
2.3 ER DIAGRAM

This Entity-Relationship Diagram (ERD) defines a comprehensive Asset Management System designed to track the lifecycle of organizational property and its interaction with personnel.

- The system is categorized into four key areas:
- Asset Lifecycle: Tracks equipment from acquisition (Purchase Record) through maintenance (Repair Record) and categorization (Asset Type).
- Personnel & Accountability: Manages Employee data and records all system activities via an Audit Log to ensure security and transparency.
- Operations: Facilitates a Borrowing Log to track asset movement between employees, including checkout and return dates.
- Facility Infrastructure: Maps assets to specific physical locations, organizing them by Building and Room (further specialized into Classrooms or Storage Rooms).



2.4 RELATIONAL SCHEMA



III. DATA IMPLEMENTATION

3.1 DDL & Data Dictionaries (DESC Table):

- Definition: DDL commands define or modify the database structure, including tables, constraints, and indexes.
- Implementation in project: created tables such as asset, borrow_log, repair_record, employee, employee_address, storage_room, audit_log, classroom, storage_room, purchase_record and type_of_asset.
- Example:

```
CREATE TABLE asset (
    asset_id INT PRIMARY KEY AUTO_INCREMENT,
    asset_name VARCHAR(50),
    asset_status VARCHAR(50),
    type_id INT,
    room_id INT,
    FOREIGN KEY (type_id) REFERENCES type_of_asset(type_id),
    FOREIGN KEY (room_id) REFERENCES room(room_id)
) ENGINE=InnoDB;
```

3.2 DML (Insert, Update, Delete):

- Definition: DML commands manipulate the data in the tables—insert, update, and delete.
- Implementation in the project:

- Insert: Adding new assets, employees, purchase records, and borrow logs.
- Update: Updating asset status, employee status, or borrow/return dates
- Delete: Removing assets, repair records, or employee records.

- **Insert example:**

```
INSERT INTO asset (asset_id, asset_name, asset_status, type_id, room_id) VALUES
(1,  'Dell PC',          'Good',         1, 2101),
(2,  'HP PC',            'Needs Repair', 1, 2102),
(3,  'Epson Projector',  'Good',         2, 3201),
(4,  'Wooden Desk',      'Good',         3, 3101),
(5,  'Office Chair',     'Good',         4, 3301),
(6,  'LG AC',            'Good',         5, 2201),
(7,  'Lenovo PC',        'Good',         1, 2101),
(8,  'Acer PC',          'Needs Repair', 1, 2102),
(9,  'BenQ Projector',   'Good',         2, 9001),
(10, 'Smart Board 55"',  'Good',         7, 2201),
(11, 'Plastic Chair',    'Good',         4, 3301),
(12, 'Samsung AC',       'Good',         5, 9003),
```

- **Update example:**

```
UPDATE asset
SET
    asset_name = assetName,
    asset_status = assetStatus,
    type_id = typeId,
    room_id = roomId
WHERE asset_id = assetId;
```

- **Delete example:**

```
DELETE FROM asset
WHERE asset_id = assetId;
```

INSERT Operations

Procedure	Table	Notes
`addNewAsset`	`asset`	Inserts a new asset record
`addNewAsset`	`audit_log`	Logs the INSERT action for `asset`
`addRepairRecord`	`repair_record`	Inserts a repair record
`addRepairRecord`	`audit_log`	Logs UPDATE for `asset` and INSERT for `repair_record`
`addPurchaseRecord`	`purchase_record`	Inserts a purchase record
`addPurchaseRecord`	`audit_log`	Logs INSERT for `purchase_record`
`addEmployee`	`employees`	Inserts employee record
`addEmployee`	`employees_address`	Inserts employee address
`addEmployee`	`audit_log`	Logs INSERT for `employees` and `employees_address`
`borrowAsset`	`borrow_log`	Inserts a borrow record if asset is available
`borrowAsset`	`audit_log`	Logs INSERT for `borrow_log`

UPDATE Operations

Procedure	Table	Notes
`updateAsset`	`asset`	Updates asset details
`updateAsset`	`audit_log`	Logs UPDATE for `asset`
`addRepairRecord`	`asset`	Updates asset status to `Good` after repair
`addRepairRecord`	`audit_log`	Logs UPDATE for `asset` (already included above)
`resignEmployee`	`employees`	Updates `resign_date`

`resignEmployee`	`audit_log`	Logs UPDATE for `employees`
`returnAsset`	`borrow_log`	Updates `return_date`
`returnAsset`	`borrow_log`	Updates `item_status` to `Returned`
`returnAsset`	`audit_log`	Logs UPDATE for `borrow_log`

DELETE Operations

Procedure	Table	Notes
`deleteAsset`	`asset`	Deletes an asset
`deleteAsset`	`audit_log`	Logs DELETE for `asset`
`deleteRepairRecord`	`repair_record`	Deletes a repair record
`deleteRepairRecord`	`audit_log`	Logs DELETE for `repair_record`
`deletePurchaseRecord`	`purchase_record`	Deletes a purchase record
`deletePurchaseRecord`	`audit_log`	Logs DELETE for `purchase_record`
`deleteEmployee`	`employees_address`	Deletes employee address
`deleteEmployee`	`employees`	Deletes employee record
`deleteEmployee`	`audit_log`	Logs DELETE for both `employees` and `employees_address`

3.3 Data Query Language (DQL)

- Definition: DQL commands are used to retrieve data from the database.
- Implementation in project: SELECT queries for reports or validations, such as employee info, assets that need repair, show records and logs.

```sql
SELECT
    a.asset_id,
    a.asset_name,
    a.asset_status,
    t.type_name,
    r.room_id,
    r.floor,
    b.building_name
FROM asset a
JOIN type_of_asset t ON a.type_id     = t.type_id
JOIN room          r ON a.room_id     = r.room_id
JOIN building      b ON r.building_id = b.building_id
ORDER BY a.asset_id;
```

SELECT Operations

Procedure	Tables	Notes
`getAssetsNeedRepair`	`asset`, `type_of_asset`	Retrieves all assets with status `'Needs Repair'` along with type and room info
`showStorageRoom`	`storage_room`, `room`	Retrieves storage room info (joined with room) by `storageId`
`getAllAssets`	`asset`, `type_of_asset`, `room`, `building`	Retrieves all assets with type, room, and building info
`showAuditLog`	`audit_log`	Retrieves all audit log entries
`showRepairRecord`	`repair_record`	Retrieves all repair records
`showPurchaseRecord`	`purchase_record`	Retrieves all purchase records
`showEmployeeRecord`	`employees`, `employees_address`	Retrieves all employee info along with address
`borrowAsset`	`asset`	Checks current room of asset (`SELECT room_id INTO currentRoom`)
`borrowAsset`	`storage_room`	Checks if asset is in a storage room

3.4 View

Definition: a virtual table created from a SELECT query. It does not store data itself; it displays data from underlying tables.

Implementation in the project:

1. asset_details:

This view shows full information about assets, including asset name, status, type, room, and building.

P**urpose**: To easily check where each asset is located and its type without writing multiple joins.

2. view_borrowed_assets:

This view displays assets that are currently borrowed by employees.

Purpose: To monitor which assets are being used and who borrowed them.

3. view_employee_building:

This view shows employees along with the building they work in.

P**urpose:** To identify employee locations in the organization.

4. view_repair_history

This view provides repair records of assets, including date, reason, and cost.

Purpose: To track maintenance history and repair expenses.

5. view_purchase_summary

This view shows purchase information for assets, including purchase date and amount.

Purpose: To manage and review asset purchasing data.

6. view_room_building:

This view displays rooms together with their building.

Purpose: To understand the structure of rooms in each building.

7. view_assets_per_room

This view counts the number of assets in each room.

Purpose: To know how many assets are stored in each room.

8. view_assets_per_building

This view counts total assets in each building.

Purpose: To analyze asset distribution across buildings.

Create view example:

```sql
-- 1.Asset detail
create view asset_details as
select a.asset_id, a.asset_name, a.asset_status , t.type_name , r.room_id , b.building_name
from asset a
join type_of_asset t on a.type_id = t.type_id
join room r on a.room_id = r.room_id
join building b on r.building_id = b.building_id;
```

- **This view use Inner Join with 4 table.**

```sql
-- 2. Borrowed Assets
create view view_borrowed_assets as
select bl.borrow_id, a.asset_name, concat(e.employee_firstname, ' ', e.employee_lastname) as full_name, bl.borrow_date
from borrow_log bl
join asset a on bl.asset_id = a.asset_id
join employees e on bl.employee_id = e.employee_id
where bl.item_status = 'Borrowed';
```

- **This view use Inner Join with 3 table.**

```sql
-- 3. Employee with Building
create view view_employee_building as
select e.employee_id, concat(e.employee_firstname, ' ', e.employee_lastname) as full_name, b.building_name
from employees e
join building b on e.building_id = b.building_id;
```

- **This view use Inner Join with 2 table.**

```sql
-- 4. Repair History
create view view_repair_history as
select r.repair_id, a.asset_name, r.fix_date, r.reason, r.repair_cost
from repair_record r
join asset a on r.asset_id = a.asset_id;
```

- **This table use Inner Join with 2 table.**

```sql
-- 5. Purchase Summary
create view view_purchase_summary as
select p.purchase_id, a.asset_name, p.purchase_date, p.amount
from purchase_record p
join asset a on p.asset_id = a.asset_id;
```

- **This table use Inner Join with 2 table.**

```sql
-- 6. Room with building
create view view_room_building as
select r.room_id, r.room_type, r.floor, b.building_name
from room r
join building b on r.building_id = b.building_id;
```

- **This table use Inner Join with 2 table.**

```sql
-- 7.view_room_managers
create view view_room_managers as
select e.employee_id, concat(e.employee_firstname, ' ', e.employee_lastname) as manager_name,  r.room_id, r.room_type
from employees e
join room r on e.managed_room_id = r.room_id;
```

- **This table use Inner Join with 2 table.**

```sql
-- 8.Total Assets per Building
create view view_assets_per_building as
select b.building_name, count(a.asset_id) as total_assets
from building b
join room r on b.building_id = r.building_id
left join asset a on r.room_id = a.room_id
group by b.building_name;
```

- **This table use Inner Join with 2 table.**
- **This table use Left Join with 2 table.**

So the last one, number 8 uses 3 Table with building is the main table.

3.5 Stored Procedure and Function

Definition: Predefined routines in the database that encapsulate logic for reuse, reduce repetitive code, and enforce rules.

Implementation in the project: Define various actions for the user on the database

Example:

```
DROP PROCEDURE IF EXISTS addNewAsset;
CREATE PROCEDURE addNewAsset(
    assetName VARCHAR(50),
    assetStatus VARCHAR(50),
    typeId INT,
    roomId INT
)
BEGIN

    INSERT INTO asset (asset_name, asset_status, type_id, room_id)
    VALUES (assetName, assetStatus, typeId, roomId);

    INSERT INTO audit_log (table_name, action, action_date)
    VALUES ('asset',  'INSERT', NOW());
END //
```

1. Employee-Related Procedures

Stored Procedure	Purpose
`addEmployee`	Add a new employee with address details.
`showEmployeeRecord`	Show all employees with address info.
`deleteEmployee`	Delete an employee record.
`resignEmployee`	Mark employee as resigned.
`getEmployeeFullname`	Get full name of an employee.
`getEmployeeInfoById`	Retrieve detailed info for a specific employee.

2. Asset-Related Procedures

Stored Procedure	Purpose
`addNewAssets`	Add a new asset to the system.
`getAllAssets`	Show all assets with room and building info.
`updateAsset`	Update asset details or status.
`deleteAsset`	Delete an asset from the system.
`getAssetsNeedRepair`	Show assets that need repair.
`showAllAssetInStorage`	List all assets currently in storage rooms.

3. Storage & Room Procedures

Stored Procedure	Purpose
`showStorageRoom`	Check storage room status and count items.

4. Borrow / Return Procedures

Stored Procedure	Purpose
`borrowAsset`	Borrow an asset and update asset status.
`returnAsset`	Return an asset and update asset status.
`showBorrowLog`	Show borrow/return history.
`showBorrowLogByAssetId`	Filter borrow/return history by asset ID.

5. Repair & Maintenance Procedures

Stored Procedure	Purpose
`addRepairRecord`	Add a repair record and update asset status.
`showRepairRecord`	Show repair history.
`deleteRepairRecord`	Delete a repair record.

---

6. Purchase Procedures

Stored Procedure	Purpose
`addPurchaseRecord`	Record a new purchase of an asset.
`showPurchaseRecord`	Show purchase history.
`deletePurchaseRecord`	Delete a purchase record.

7. Audit / Logging Procedures

Stored Procedure	Purpose
`showAuditLog`	Show actions performed in the system (asset, employee, repair, borrow, etc.).

IV. CONCLUSION AND FUTURE WORK

4.1 Outcome / Completed Work

The project successfully implemented an Asset Management Database System designed to manage assets, employees, storage rooms, repair records, and borrowing transactions. The system was developed using a relational database model with properly defined tables, relationships, and constraints.

The database includes multiple entities such as assets, rooms, storage rooms, employees, repair records, purchase records, and borrow logs. Core database operations were implemented using DDL, DML, and DQL, while business logic was handled using stored procedures. These procedures allow users to perform operations such as adding assets, borrowing and returning assets, updating asset status, tracking repairs, and managing employee information.

Additionally, an audit logging mechanism was implemented to record important operations such as insertions, updates, and deletions. This helps maintain accountability and track system activities.

Overall, the system provides an organized way to manage assets and track their status, location, and usage history.

4.2 Strong Points

Structured database design: The system uses normalized tables with clear relationships between entities such as assets, employees, and rooms.
Use of stored procedures: Key operations such as borrowing assets, returning assets, and updating records are handled through stored procedures, improving consistency and maintainability.

Audit logging: The audit log helps track system changes and increases transparency.
Data integrity: Foreign key constraints ensure valid relationships between tables.
Functional coverage: The system supports asset tracking, repair management, employee management, and storage management.

4.3 Weak Points

Limited validation logic: Some procedures rely on basic checks and could include more advanced validation rules.
User interface not implemented: The system currently operates directly through SQL queries rather than a graphical interface.
Limited reporting capabilities: While data can be queried, advanced analytics or dashboards are not implemented.
Scalability considerations: The current implementation is suitable for a small to medium dataset but may require optimization for larger systems.

4.4 Challenges / Difficulties

During the development of this project, several challenges were encountered:

Database relationship design: Ensuring proper relationships between assets, rooms, storage rooms, and employees required careful planning.
Stored procedure logic: Implementing procedures for borrowing, returning, and updating assets required handling multiple tables simultaneously.
Error handling: Managing conditions such as preventing assets from being borrowed if they are not in storage required additional logic.
Data consistency: Maintaining consistent asset status across borrow logs, repair records, and storage location required careful updates.

Despite these challenges, the issues were resolved through testing and iterative improvements to the database design.

4.5 Future Work

Several improvements can be implemented in future versions of the system:

- User Interface Integration: Develop a web or desktop application that interacts with the database.
- Advanced reporting: Add analytical reports such as asset usage statistics, repair frequency, and cost tracking.
- Improved validation: Implement stronger validation rules in stored procedures to prevent invalid transactions.
- Inventory capacity control: Automatically detect when storage rooms reach capacity.
- Automated alerts: Notify administrators when assets require repair or have been borrowed for too long.
- Performance optimization: Add indexing and optimize queries for handling larger datasets.

These enhancements would make the system more scalable, user-friendly, and suitable for real-world asset management environments